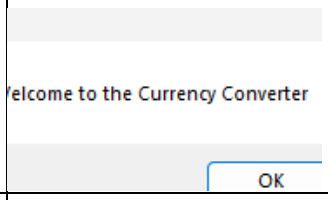
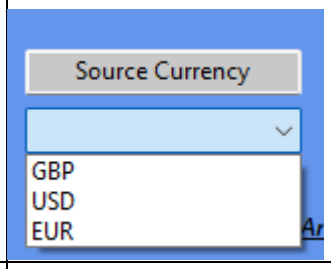
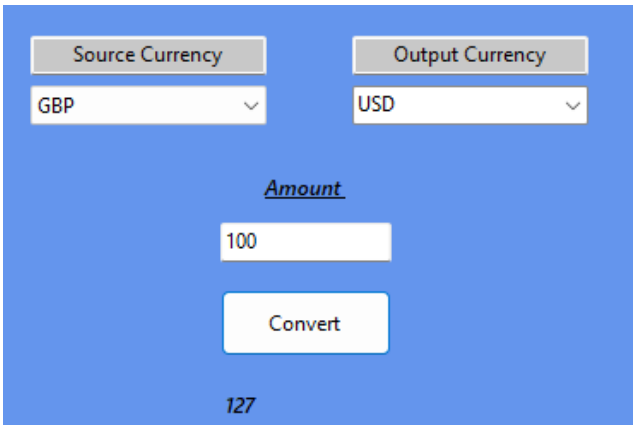
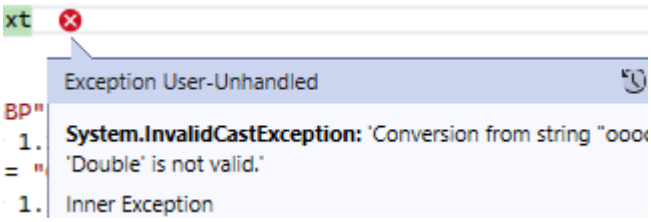
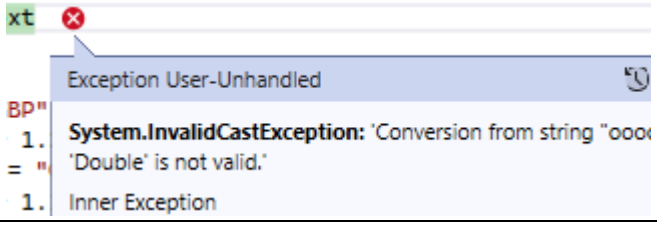
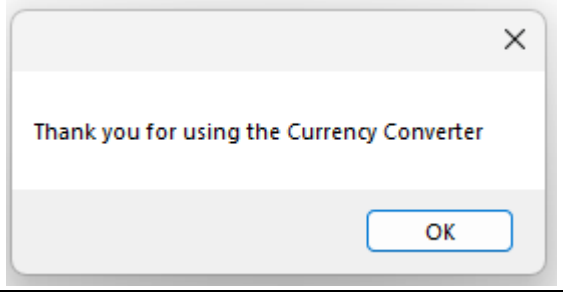
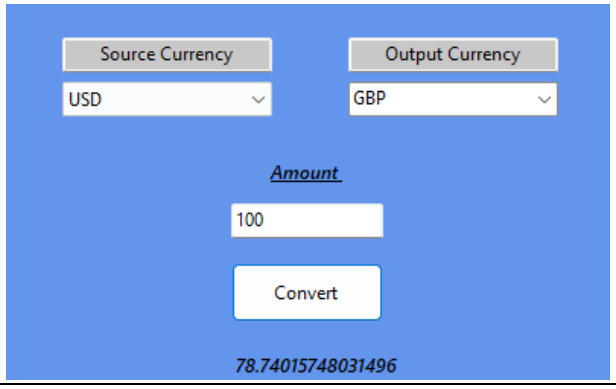
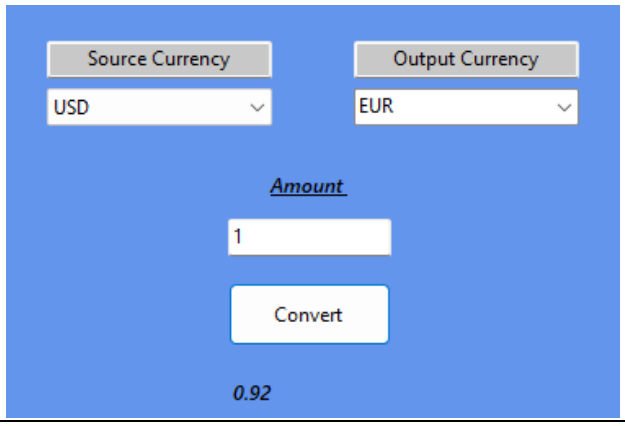
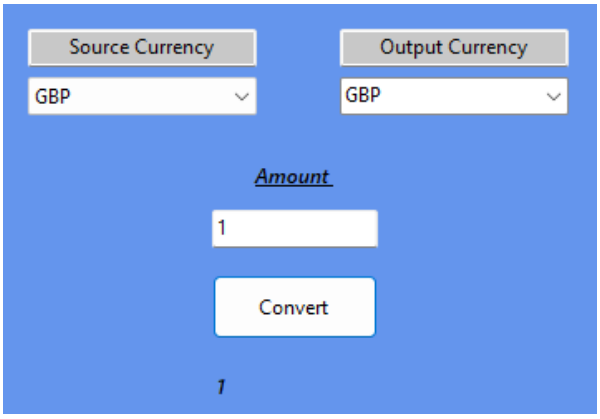
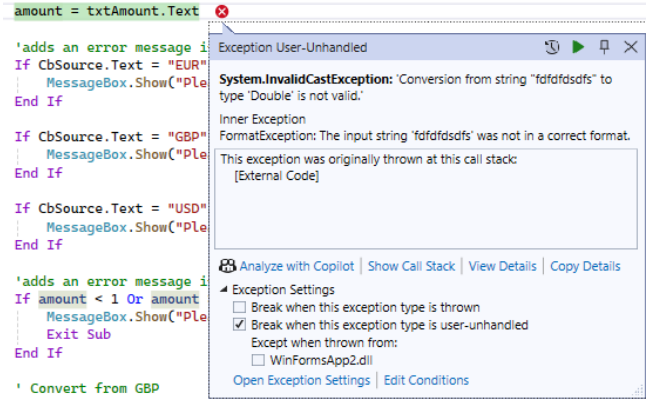
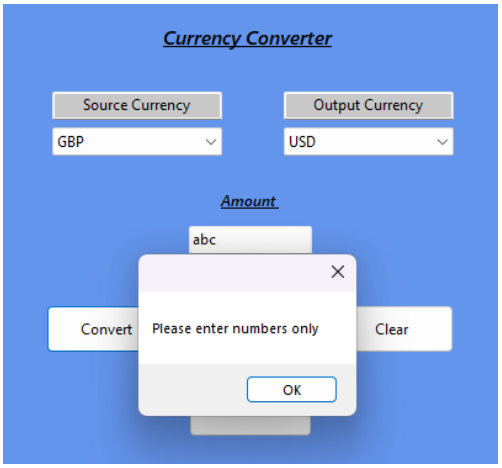
| Test Number | Description | Excepted Results | Actual Results | Corrective action |
|---|---|---|---|---|
| 1 | Welcome message appears when program runs | Message box loads | Pass   | None |
| 2 | Combo boxes items populate when form loads | Items populated | Pass   | None |
| 3 | 100 GBP to USD conversion | 127USD | Pass   | None |
| 4 | User enters non-numeric values into the conversion box | Message Box – warning user to enter a sensible value | Fail   | Add an input check to btnConvert_Click |
| 5 | User enters nothing into the conversion box | Message Box – warning user to enter a sensible value | Fail   | Add an input check to btnConvert_Click |
| 6 | User exits the program | A confirmation message appears for the user | Pass   | None |

| | | | | |
|---|---|---|---|---|
| 7 | When converting between different currencies | USD to GBP would equal a conversion rate of 0.79 | Pass<br>  | None |
| 8 | When converting between different currencies | USD to Euros would equal a conversion rate of 0.92 | Pass<br>  | None |
| 9 | When converting between the same currency | Message Box – warning user to select two different currencies | Fail<br>  | Add an input check ensuring the user selects two different currencies to convert |

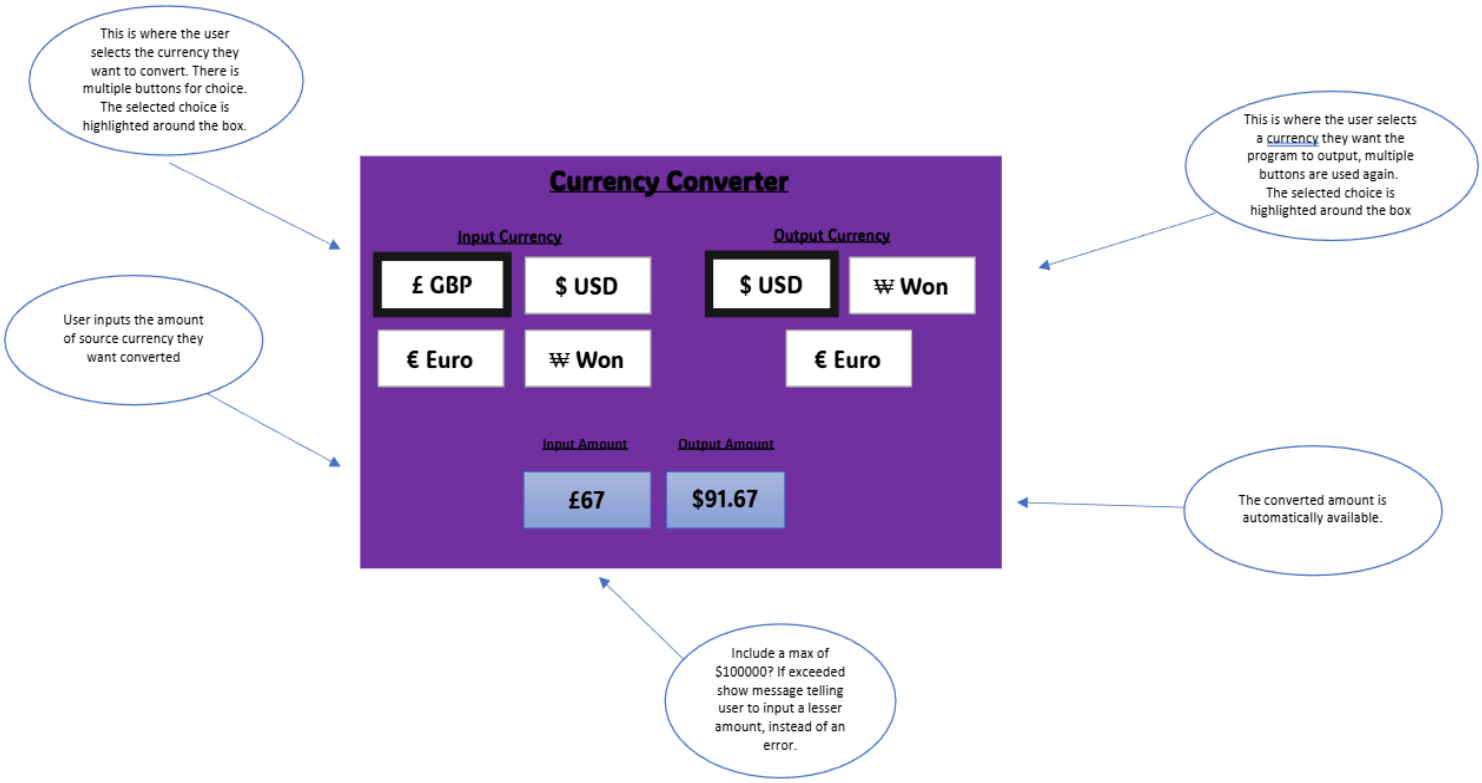
| | | | | |
|---|---|---|---|---|
| 10 | User selects an extremely high number | Message Box – warning user to select a reasonable number |  Pass | |
| 11 | User selects a value outside of the range check (1-100000) | Message Box – warning user to select a higher number | Pass<br>  | |
| 12 (USER FOUND) | User typed in letters instead of numbers | Message Box – warning user to insert numeric values instead of letters | Fail<br>  | Add an input check ensuring the user uses numeric values instead of letters |
| 13 | User typed in letters | | Fixed | Added the input check ensuring |

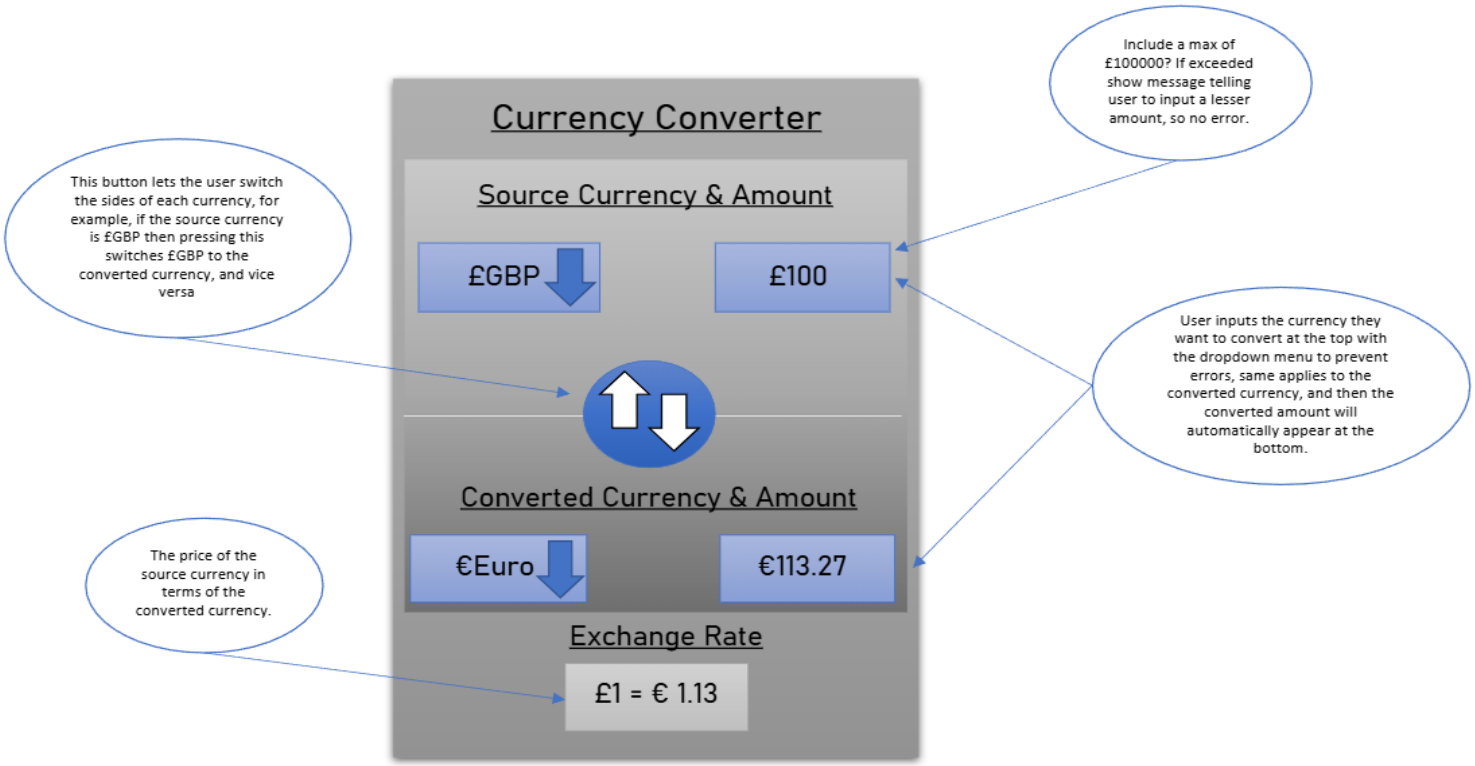
| | | | | |
|---|---|---|---|---|
| (USER FOUND) | instead of numbers | |  <br>`If Not Double.TryParse(txtAmount.Text, amount) Then`<br>`  MessageBox.Show("Please enter numbers only")`<br>`  Exit Sub`<br>`End If` | the user uses numeric values instead of letters |

## Worksheet 8

**Alternative Screen Design (s) and Screen Flow**

Brief outline of alternative solutions for the intended software program, e.g. for screen layout and navigation:






My first alternative design has a purple interface where users select their input and output currencies using a set of dedicated buttons. To improve navigation and clarity, the selected currency is highlighted with a border once clicked. This design is built for safety as it has a maximum input limit of 100000 if a user tries to enter a higher amount, the program triggers a message box instead of crashing with an error.

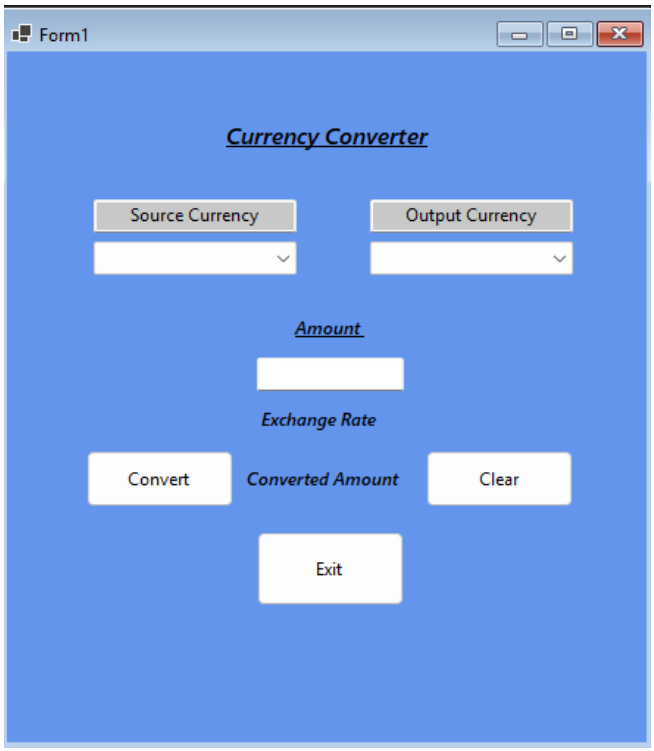
The second design offers a more complex grey interface that uses dropdown menus for currency selection to save space and prevent input errors. A unique feature of this layout is the Switch button that allows users to instantly swap the source and converted currencies, also this version displays the live exchange rate at the bottom and automatically updates the converted amount as the user types, while keeping the same 100000 limit as the first design.

**Explanation as to why alternative designs ideas were rejected:**

I rejected these because they did not balance being simple with being functional. I turned down the first design because having separate buttons for every currency takes up too much screen space, which would make the UI look cluttered and difficult to navigate if I were to add more currencies in the future. I rejected the second design because features like the Switch button added extra complexity that some people may find confusing. Since my goal was to create an easy to use tool that anyone can use, I chose a simpler final design for the best balance

## Rationale for Design Decisions

Justification for chosen designs with regard to user requirements and identification of any constraints:




My layout focuses on a clean blue theme that keeps everything organised in one column. I chose to use dropdown menus for selecting both the source and output currencies. This is a key design decision because it prevents users from making typing errors when choosing a currency which makes the program more reliable. It also keeps the screen from getting cluttered with buttons, making it very easy for someone to understand.

In the middle of the screen, ive put a clear input box where the user can type the amount they want to convert. Previously below this, I had the convert button, but in light of user feedback, I've interested an exchange rate and clear button below here instead, so I moved the convert button to the right. I chose to include a convert button specifically so that the user has total control over when the calculation happens, rather than it changing constantly while they are still typing like one of my alternative designs. The final converted amount then appears in its own box at the bottom, here it keeps the input and the result separate so they'r easy to read.

I have put a limit of 100000 for the conversion so if a user tries to enter a number higher than this the program will show a helpful message asking them to put in a smaller amount instead of just crashing or showing a technical error.

I've placed an exit button to make it more convenient. This button gives the user an easy way to close the program properly whenever they're finished, and I've also included a message box thanking the user for using the program once they click the exit button.

**Worksheet 9 - Review**

Completed review, including screenshots, to evaluate the results of the testing and how well the program fits its intended purpose and meets the original requirements.

1. **The extent to which the software program is fit for purpose and meets the original requirements.**

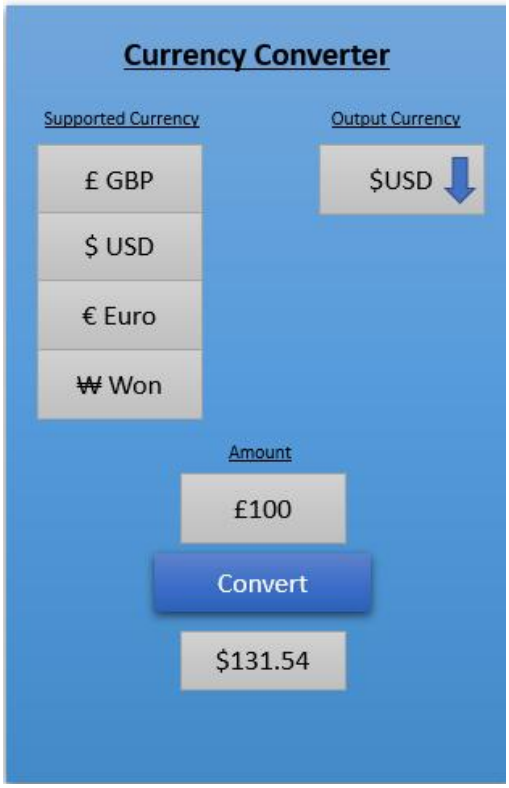
   The original requirements were:

Convert GBP to foreign currency
Convert foreign currency to GBP
Allow user to alter exchange rates
Tested and refined

The program was designed to calculate currency exchange rates for customers. My final program allows the user to enter an amount in British pounds and convert it into another currency. It also allows conversion from a foreign currency back into British pounds, such as Euros which meets the requirement of two way conversions.

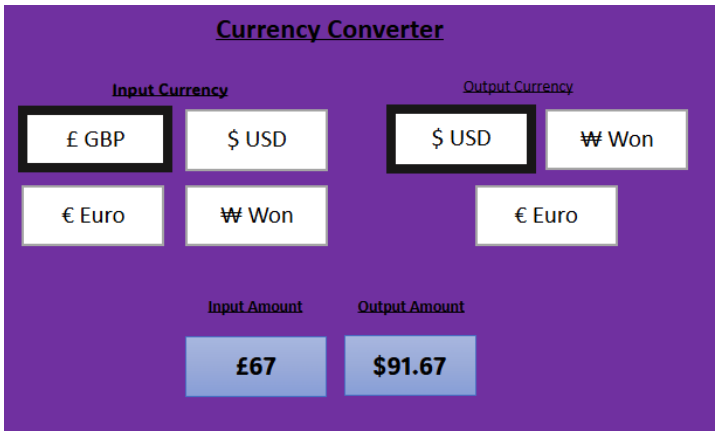
The interface is simple and easy to use so people can complete conversions quickly. I have added Input validation to prevent letters being entered into the amount field or very high numbers.

However, my program is limited to a small number of currencies and does not automatic update rates from the internet, so it is limited.

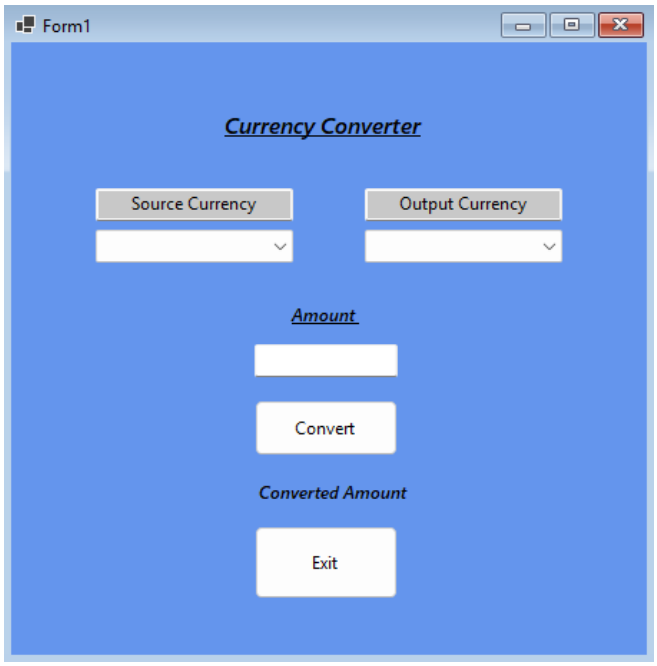



**Evaluation of the initial designs against the completed program.**

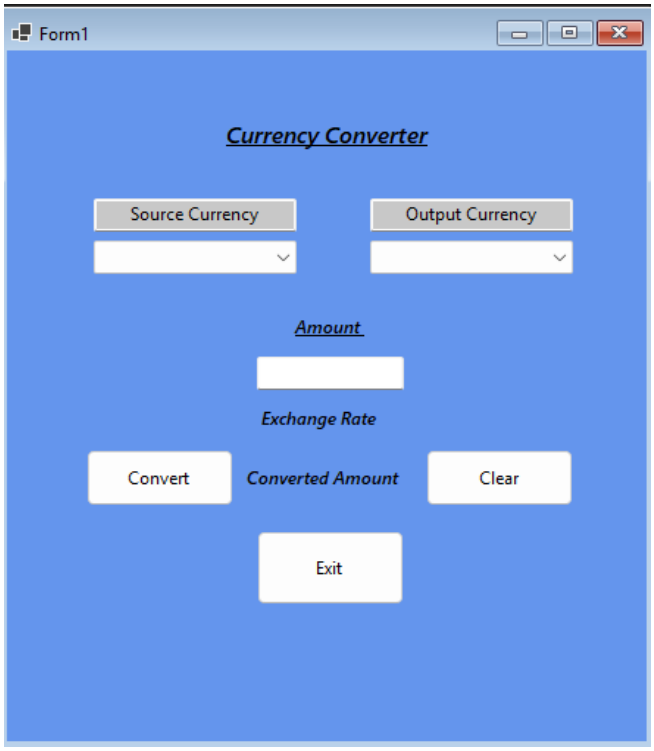
**Initial Designs:**



**Final design before improvements**




**Final design after improvements**




When comparing my first design ideas with the final version of the program, I did add most features into my completed software. In my first draft I focused on a layout with dropdown menus to select the source currency, the output currency and an input box for the amount. The overall structure of the finished program still follows this original idea.

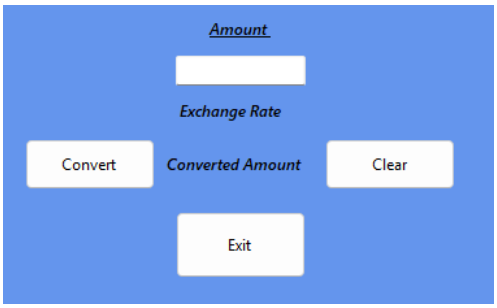
In my second draft, I experimented with using different selection boxes for each currency. While this made the options clearly visible, it would take up more space and make it harder to add more currency later. In my final version I used drop down menus / combo boxes for the source and output currencies like my first draft, to make the interface more organised and reduce the clutter. Also, if more currencies are to be added later then this makes the program more flexible and easier to maintain.

So my completed program keeps the core layout from my early design, but it improves the usability of the interface.

1. **Changes that could be made in the light of user feedback.**

   After asking a classmate to test my program, what they said was that "your program is easy to use but it wasn't obvious to me what the current exchange rate was, it would be clearer if there was a visible label showing the rate being used. Also a Clear button would also be really useful to reset all everything quickly. And It would be nice if it supported more currencies or even let me add my own currencies.

   Taking this feedback on board, I went and added an exchange rate label so the person can clearly see the rate being used for the calculation, it automatically updates as you change the source currency. I also added a Clear button to quickly reset all the input and output fields. The suggestion to add more to the currency list is something I don't think is needed right now.


2. **Changes made from the design stage. Justification of these changes in terms of the original requirements and the features of the language used, and any other constraints.**

The general design of my program stayed similar to my first original design, as both versions used drop-down boxes for currency selection and an input box for the amount.

However, improvements were made during development. I added input validation using the KeyPress event set to off to prevent letters being entered into the amount field. I didn't show this in the original design, but it was needed to reduce user errors and improve reliability.

But I did scrap my second design idea which used individual boxes for each currency. This made all options visible at once, but it would've taken up more space on the form and made the interface look cluttered. And it would have been difficult to use if more currencies needed to be added, so instead using drop down boxes makes the program more organised and more scalable if I were to add more currencies

3. **<u>Three recommendations for how the completed program could be improved to ensure it is fully functional, well coded and fit for purpose.</u>**

1. Change IF statements into case statements for efficiency, like converting currencies or checking input, to make it more performative and easier to read while doing the same thing. (Added)

```
Select Case conversionKey
    Case "GBP_USD"
        result = amount * 1.27
    Case "GBP_EUR"
        result = amount * 1.17
    Case "USD_GBP"
        result = amount / 1.27
    Case "USD_EUR"
        result = amount * 0.92
    Case "EUR_GBP"
        result = amount / 1.17
    Case "EUR_USD"
        result = amount / 0.92
    Case Else
        result = amount
End Select
```

2. Include a button to reset the amount and result boxes so the person can start a new conversion quickly without closing the program. (Added)

```
    Private Sub Button1_Click(sender As Object, e As EventArgs) Handles btnClear.Click
        txtAmount.Text = ""
        CbSource.SelectedIndex = -1
        CbOutput.SelectedIndex = -1
        lblResult.Text = ""
        lblRate.Text = ""
    End Sub
End Class
```

3. Ensure the converted amount is displayed neatly by being rounded to a reasonable number of decimals, like 50.98 instead of 50.9884192. This makes the result much easier to read by preventing long strings. (Added)

```
    lblResult.Text = result.ToString("0.00")
End Sub
```